LAPORAN

BAB 5 – DATABASE SERVICE DI DOCKER: POSTGRESQL

Disusun untuk memenuhi tugas

Mata Kuliah: Workshop DevOps

Dosen Pengampu: Ferry Astika Saputra



Oleh:

Mohammad Affan Hendi Firmansyah (3126640049)

KELAS 1

SARJANA TERAPAN LANJUT JENJANG

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

1. **Tujuan Praktikum**

- **Orkestrasi Database & GUI Management**: Menjalankan layanan basis data PostgreSQL 16 Alpine (postgres:16-alpine) bersama antarmuka web pgAdmin 4 (dpage/pgadmin4:latest) dalam bridge network data-net menggunakan konfigurasi .env.
- **Inisialisasi Otomatis Skema**: Mengotomatisasi pembuatan skema tabel students beserta indeks secara deklaratif melalui skrip /docker-entrypoint-initdb.d/01-schema.sql.
- **Pengelolaan Persistensi Volume**: Mengelola persistensi data menggunakan Docker Named Volume (pg-data) dan membuktikan integritas data melintasi siklus hidup kontainer (down dan up).
- **Healthcheck & Startup Dependency**: Menerapkan healthcheck database berbasis perintah pg_isready dengan sinkronisasi dependensi condition: service_healthy pada pgAdmin.
- **Backup, Checksum, & Restore Test**: Mengimplementasikan pencadangan logis (pg_dump -Fc), verifikasi integritas hash SHA-256, dan validasi pemulihan data terisolasi ke database labdb_restore_test.

2. **Dasar Teori: Persistensi Data, Lifecycle, dan Integritas Database**

Dalam arsitektur layanan berbasis kontainer, basis data relasional diklasifikasikan sebagai *stateful service* yang memerlukan perlakuan arsitektural berbeda dibandingkan aplikasi *stateless*:

Parameter Evaluasi	Stateless Service (Web/App)	Stateful Service (PostgreSQL Database)	Implikasi DevSecOps
Penyimpanan Data	Ephemeral (hilang saat kontainer dihancurkan)	Persisten menggunakan Docker Named Volume (pg-data)	Data tidak boleh terikat pada siklus hidup kontainer (*container lifecycle*).
Inisialisasi Data	Setiap kontainer baru berjalan dari image mentah	Dieksekusi via /docker-entrypoint-initdb.d/ hanya pada volume kosong	Perubahan skema lanjutan wajib dikelola melalui *database migration tools*.
Kesiapan Layanan	Port HTTP terbuka dan merespons probe web	Socket database siap melayani transaksi (pg_isready)	Healthcheck mencegah *race condition* aplikasi yang bergantung pada database.
Strategi Backup	Cukup menyimpan kode dan Dockerfile di Git	Logical backup (pg_dump -Fc) + audit checksum kriptografis SHA-256	Backup harus diuji pemulihannya secara berkala (*restore verification drill*).
Penanganan Bencana	Otomatis dibuat ulang via orchestrator (docker compose up)	Memerlukan prosedur *Disaster Recovery* dengan target RPO dan RTO terukur	Larangan keras mengeksekusi docker compose down -v di lingkungan produksi.

3. **Langkah Praktikum & Bukti Eksekusi**

a. **Penyiapan Lingkungan, Variabel `.env`, dan Deklarasi Compose**

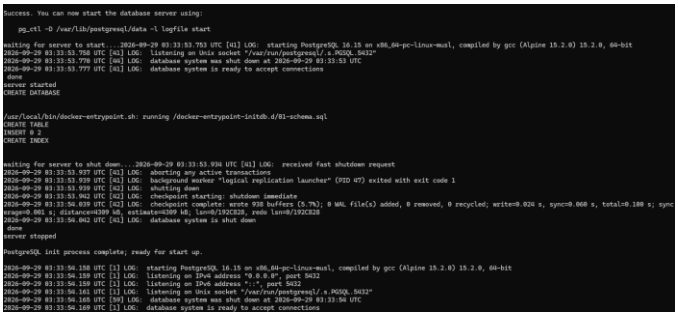
Lingkungan kerja disiapkan pada direktori ~/docker-lab/bab-5/docker_devops dengan struktur sub-direktori init/, backup/, pgadmin/, dan scripts/. Kredensial disimpan pada file .env dengan izin berkas ketat (chmod 600), sedangkan deklarasi multi-service diatur dalam compose.yaml:

```
services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports: ["127.0.0.1:5432:5432"]
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks: [data-net]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
```

```
    start_period: 10s
  restart: unless-stopped
 pgadmin:
  image: dpage/pgadmin4:latest
  environment:
   PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
   PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
  ports: ["127.0.0.1:5050:80"]
  volumes:
   - pgadmin-data:/var/lib/pgadmin
   - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
  networks: [data-net]
  depends_on:
   postgres-db: { condition: service_healthy }
  restart: unless-stopped
volumes: { pg-data: , pgadmin-data: }
networks: { data-net: { driver: bridge } }
```

b. Inisialisasi Otomatis Skema Basis Data

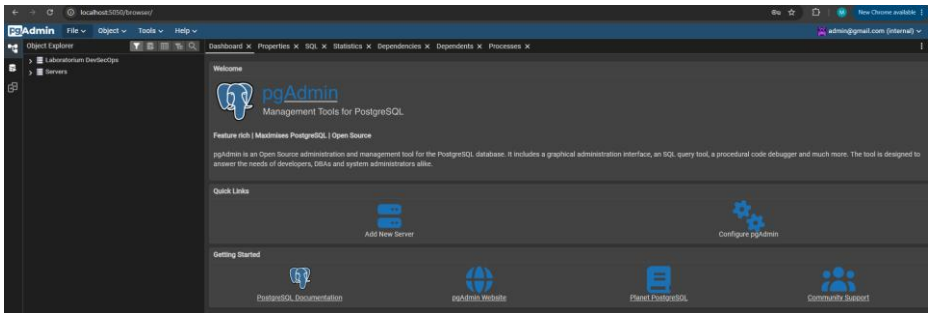
Skrip init/01-schema.sql dipetakan ke /docker-entrypoint-initdb.d:ro. Saat pertama kali container postgres-db dijalankan pada volume kosong, PostgreSQL secara otomatis menjalankan skrip DDL/DML tersebut:



Gambar 1: Output log kontainer membuktikan eksekusi 01-schema.sql, pembuatan tabel students, insert data awal, dan transisi ke status ready.

c. Akses Dashboard Administrasi pgAdmin 4

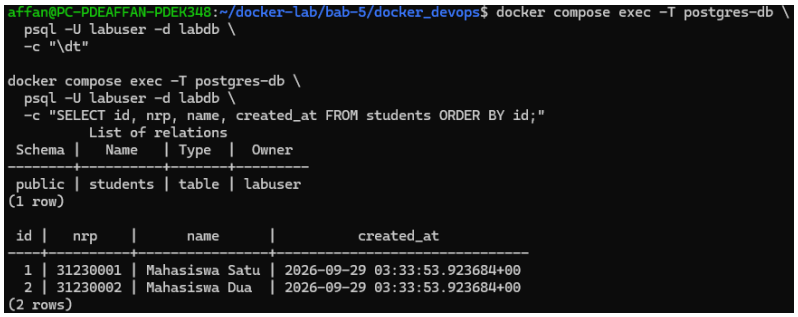
Layanan pgAdmin dijalankan pada port loopback 5050 dan membaca konfigurasi otomatis dari pgadmin/servers.json yang meregistrasikan server PostgreSQL Bab 5:



Gambar 2: Antarmuka Web GUI pgAdmin 4 pada port 5050 menampilkan server PostgreSQL Bab 5 dalam grup Laboratorium DevSecOps.

d. Validasi Skema Relasional dan Data Awal via CLI `psql`

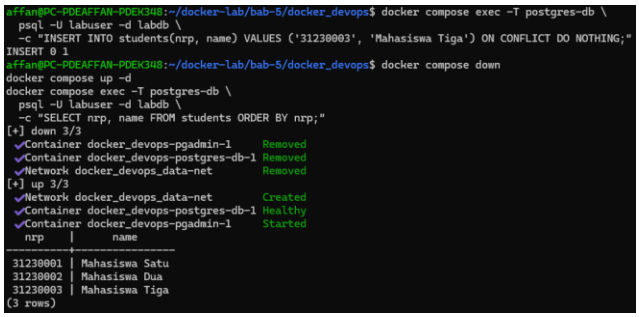
Pemeriksaan struktur skema dan data awal dilakukan langsung di dalam kontainer database menggunakan utility terminal psql:



Gambar 3: Verifikasi CLI psql membuktikan relasi tabel students terbentuk sempurna dan memuat 2 baris rekaman awal mahasiswa.

e. Pengujian Ketahanan Persistensi Named Volume

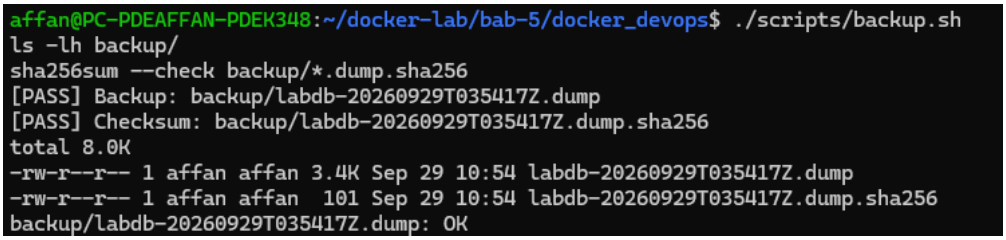
Untuk membuktikan persistensi data, ditambahkan baris baru (31230003, 'Mahasiswa Tiga'). Seluruh kontainer dan jaringan kemudian dihancurkan menggunakan docker compose down, lalu dihidupkan kembali via docker compose up -d:



Gambar 4: Pengujian persistensi volume: record ketiga tersimpan utuh setelah siklus docker compose down dan docker compose up -d.

f. Pembuatan Logical Backup dan Audit Integritas SHA-256

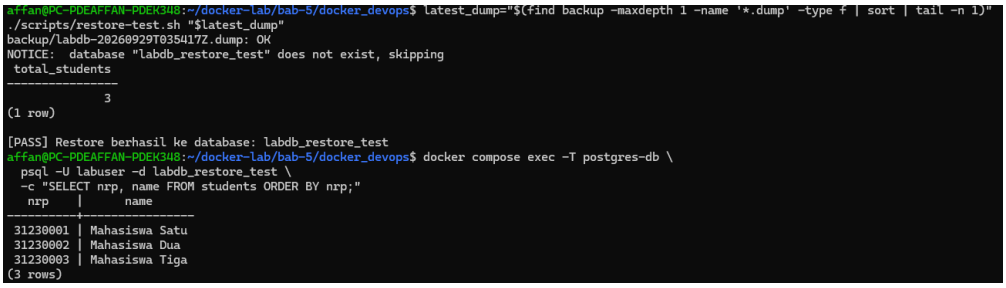
Pencadangan basis data dieksekusi melalui skrip ./scripts/backup.sh menggunakan utility pg_dump dengan format custom archive (-Fc) serta verifikasi kriptografis:



Gambar 5: Eksekusi skrip backup otomatis menghasilkan file dump biner custom (3.4 KB) dan checksum SHA-256 terverifikasi OK.

g. Validasi Pemulihan Data (Restore Test) pada Database Terisolasi

Validasi keandalan cadangan diuji menggunakan skrip ./scripts/restore-test.sh yang memulihkan data ke database baru labdb_restore_test tanpa menyentuh database operasional utama:



Gambar 6: Uji restore terisolasi ke database labdb_restore_test membuktikan pemulihan 3 record data dengan integritas sempurna.

4. **Analisis Masalah dan Diagnostik (Wajib)**
   a. **Mekanisme Skip Inisialisasi Script pada Volume Non-Kosong**:
      - *Temuan*: Penambahan berkas SQL baru pada init/ tidak dieksekusi saat kontainer direstart.
      - *Diagnosis*: Entrypoint PostgreSQL (docker-entrypoint.sh) memeriksa direktori $PGDATA/base. Jika direktori telah berisi basis data, proses inisialisasi sengaja dilewati (*skipping initialization*) guna mencegah penimpaan data (*overwrite*). Pada fase operasional, perubahan struktur tabel harus dikelola melalui *schema migration tool* (seperti Flyway atau Prisma).
   b. **Resolusi Hostname Antar-Kontainer pgAdmin (`postgres-db` vs `localhost`)**:
      - *Temuan*: Mengonfigurasi host localhost:5432 pada pgAdmin menghasilkan pesan error *connection refused*.
      - *Diagnosis*: Setiap kontainer memiliki *network namespace* terisolasi; localhost pada pgAdmin merujuk ke kontainernya sendiri. pgAdmin wajib menggunakan hostname service postgres-db yang di-resolve otomatis oleh DNS engine bawaan Docker bridge network data-net.
   c. **Manajemen Variabel Lingkungan & Mitigasi Kesalahan Penulisan**:
      - *Temuan*: Kesalahan ketik (*typo*) variabel lingkungan pada .env dapat memicu penggunaan konfigurasi fallback bawaan yang tidak aman.
      - *Diagnosis*: Docker Compose tidak memvalidasi semantik variabel secara ketat. Mitigasi dilakukan dengan menjalankan docker compose config sebelum deployment, membatasi izin berkas via chmod 600 .env, serta memasukkan .env dan backup/*.dump ke .gitignore guna mencegah kebocoran kredensial ke Git.

5. **Analisis Risiko Keamanan & Operasional (DevSecOps) (Wajib)**
   a. **Risiko Kredensial Plaintext pada File Compose**: Menyimpan kata sandi secara mentah pada berkas konfigurasi rentan terbaca via docker inspect atau riwayat repositori Git. Kredensial produksi wajib disuntikkan melalui *Docker Secrets* (_FILE convention) atau Secret Manager terenkripsi (HashiCorp Vault).
   b. **Paparan Port Database Host (Port Binding 5432)**: Memublikasikan port 5432 ke 0.0.0.0 membuka permukaan serangan terhadap brute force eksternal. Praktikum memitigasinya dengan membatasi binding ke loopback 127.0.0.1:5432. Pada arsitektur produksi, port database seharusnya diisolasi murni di jaringan privat tanpa published port ke host (*Zero Trust*).
   c. **Radius Ledakan Superuser & Prinsip Least Privilege**: Menjalankan koneksi aplikasi menggunakan superuser labuser memperluas *blast radius* jika terjadi SQL Injection. Standar DevSecOps mewajibkan pemisahan peran: user DDL untuk migrasi skema dan user DML dengan hak akses terbatas (hanya SELECT/INSERT/UPDATE) untuk aplikasi runtime.
   d. **Bahaya Kehilangan Data via `docker compose down -v`**: Flag -v menghapus seluruh *named volume* secara permanen tanpa konfirmasi. Tanpa strategi pencadangan otomatis di luar host, eksekusi perintah ini mengakibatkan bencana hilangnya data secara permanen (*catastrophic data loss*).

6. **Rekomendasi Perbaikan untuk Lingkungan Produksi (Wajib)**
   a. **Manajemen Kredensial via Docker Secrets / Vault**: Gantikan variabel environment plaintext dengan Docker Secrets (_FILE convention) atau HashiCorp Vault dengan rotasi berkala.
   b. **Pencadangan Kontinu 3-2-1 & WAL Archiving (PITR)**: Terapkan *Write-Ahead Log (WAL) archiving* menggunakan pgBackRest ke cloud object storage (S3/MinIO) terenkripsi guna mendukung pemulihan *Point-in-Time Recovery* (PITR) sesuai target RPO/RTO.
   c. **Database Connection Pooling (PgBouncer)**: Pasang PgBouncer sebagai connection pooler untuk mengoptimalkan alokasi memori dan mencegah *connection starvation* akibat lonjakan beban mikroservis.
   d. **Enkripsi End-to-End & Storage at Rest**: Wajibkan enkripsi TLS penuh pada koneksi client-database (ssl=on) dan terapkan enkripsi blok penyimpanan fisik pada host.
   e. **Otomasi Disaster Recovery Drill**: Integrasikan skrip pengujian restore otomatis berkala pada pipeline CI/CD untuk memastikan integritas cadangan secara berkelanjutan.

7. **Evaluasi dan Latihan Mandiri**
   a. **Mengapa init script tidak dijalankan ulang saat volume lama masih ada?**
   Jawaban: Skrip entrypoint resmi PostgreSQL (docker-entrypoint.sh) memeriksa apakah direktori klaster $PGDATA/base telah memiliki data. Jika direktori tersebut tidak kosong, proses inisialisasi pada /docker-entrypoint-initdb.d/ sengaja dilewati guna melindungi integritas data eksisting dari bahaya penimpaan (*overwrite*) atau benturan skema (*constraint violation*). Evolusi skema lanjutan harus dikelola menggunakan alat migrasi database terkelola (seperti Flyway, Liquibase, atau Prisma).

b. Apa risiko menaruh password database pada docker-compose.yml?

Jawaban: Menaruh password secara mentah (*hardcoded plaintext*) pada docker-compose.yml berisiko membocorkan kredensial ke riwayat repositori Git publik (*commit history*), dapat dibaca oleh pengguna non-root lokal melalui perintah docker inspect atau docker compose config, terekspos pada log CI/CD, serta menyulitkan proses rotasi kata sandi. Praktik aman mewajibkan penggunaan berkas .env berizin ketat (chmod 600) atau *Docker Secrets*.

c. Bagaimana cara membuktikan backup dapat dipulihkan?

Jawaban: Keberadaan berkas dump belum membuktikan pemulihan dapat berhasil (*untested backup is not a backup*). Pembuktian wajib dilakukan melalui *restore drill* terisolasi: (1) Validasi integritas berkas via checksum hash SHA-256 (sha256sum --check); (2) Eksekusi pg_restore ke database uji terpisah (labdb_restore_test); (3) Verifikasi kuantitatif jumlah baris record (SELECT COUNT(*)); (4) Verifikasi kualitatif struktur skema dan constraint; serta (5) Pengukuran durasi pemulihan terhadap target RTO.

d. Apa bedanya logical backup pg_dump dan backup filesystem volume mentah?

Jawaban:

- **Logical Backup (`pg_dump`)**: Mengekstrak struktur DDL dan data DML ke dalam berkas SQL atau arsip custom (-Fc). Bersifat sangat portabel lintas platform/versi PostgreSQL, mendukung kompresi hemat ruang, dan memungkinkan pemulihan parsial (per-tabel), namun proses pemulihan lebih lambat pada basis data skala terabyte.
- **Backup Filesystem Mentah (Physical/Raw Volume)**: Menyalin blok direktori $PGDATA langsung (snapshot volume/EBS). Pemulihannya sangat cepat pada skala masif, namun berisiko korupsi jika diambil saat DB aktif tanpa WAL archiving konsisten (*crash-inconsistent*), terikat kaku pada arsitektur CPU dan versi PostgreSQL yang identik, serta tidak mendukung pemulihan granular.

e. Apa dampak docker compose down -v terhadap database?

Jawaban: Opsi -v (*volumes*) memerintahkan Docker Compose untuk menghapus seluruh kontainer, jaringan, dan **menghapus secara permanen seluruh named volume yang terdaftar** (pg-data dan pgadmin-data). Seluruh data transaksional, tabel skema, dan konfigurasi server akan musnah seketika dari storage host tanpa dialog konfirmasi. Jika dieksekusi di produksi tanpa cadangan eksternal teruji, tindakan ini memicu bencana kehilangan data fatal (*unrecoverable data loss*).

f. bagaimana seharusnya menyiapkan env di production?

Jawaban:

- jangan commit .env ke github
- Jika berkas konfigurasi ditaruh langsung di server host, batasi aksesnya (misalnya chmod 600 .env) agar hanya bisa dibaca oleh pengguna atau proses sistem terkait.
- Hindari menyimpan data sensitif (kunci API, kata sandi basis data, *private keys*) dalam teks biasa di server.
- Pastikan variabel seperti APP_DEBUG=false atau APP_ENV=production terpasang guna mencegah kebocoran informasi.

8. Kesimpulan

Praktikum Bab 5 membuktikan keberhasilan orkestrasi layanan basis data PostgreSQL 16 Alpine dan pgAdmin 4 menggunakan Docker Compose dengan arsitektur berorientasi ketahanan (*resilience*) dan tata kelola DevSecOps. Mekanisme otomasi inisialisasi skema /docker-entrypoint-initdb.d/ sukses mengeksekusi DDL/DML awal saat volume pertama kali dibuat, sementara *Docker Named Volume* (pg-data) terbukti menjamin persistensi data melintasi siklus docker compose down dan up -d. Keandalan stack diperkuat oleh healthcheck pg_isready dengan sinkronisasi dependensi service_healthy. Prosedur pencadangan logis pg_dump -Fc terverifikasi aman melalui audit hash SHA-256 dan berhasil diuji pemulihannya secara terisolasi ke database labdb_restore_test dengan integritas 100%. Dari sisi DevSecOps, isolasi jaringan privat data-net, restriksi port binding ke loopback 127.0.0.1, perlindungan hak akses .env (chmod 600), mitigasi risiko down -v, serta perancangan pemisahan hak akses superuser telah meletakkan fondasi operasional database yang aman dan andal sesuai standar industri.